

The Computer Use Control Stack

A field guide for routing desktop tasks through the cheapest, most reliable interface first: commands, app APIs, accessibility trees, keyboard pipelines, then local vision.

```
intent -> choose lane -> execute one typed action -> verify state  
-> recover or escalate
```

The desktop DOM

The operating system accessibility tree: roles, names, values, state, actions, and bounds.

Pixels and coordinates

Pixels are an observation. Coordinates are an actuator target. Neither explains what a control means.

CLI-first goal

Do not claim "40% of all tasks." Measure your own workload, then push the eligible command lane toward complete coverage.

Vim correction

Blind key mashing is weak. A named, observable, verified Vim-style macro is a powerful automation node.

1

API / CLI

2

DOM / a11y

3

Keyboard node

4

Local vision

5

Raw input

Use the richest interface the software exposes

Reliability falls as meaning disappears. Route each task from semantic control toward visual control, not the other way around.

S
Native API / CLI
Structured commands

Application APIs, app-native command line interfaces, scripts, direct file formats, database calls, process control.

Best: speed, repeatability, audibility

A
DOM / a11y tree
Semantic elements

Browser Document Object Model (DOM), Windows UI Automation, macOS Accessibility, Linux AT-SPI. Select by role/name/state and invoke actions.

Best: GUI control without visual guessing

B
Keyboard nodes
Deterministic sequences

Hotkeys, command palettes, Vim motions, tmux macros, launcher flows. Strong when focus and postconditions are observable.

Best: fast composable workflows

C
Local vision
Semantic blind spots

Screenshot, optical character recognition (OCR), icon/text detection, visual grounding, then click coordinates. Use only after semantic lanes miss.

Best: canvas, games, remote desktops

D
Raw pixels
Last resort

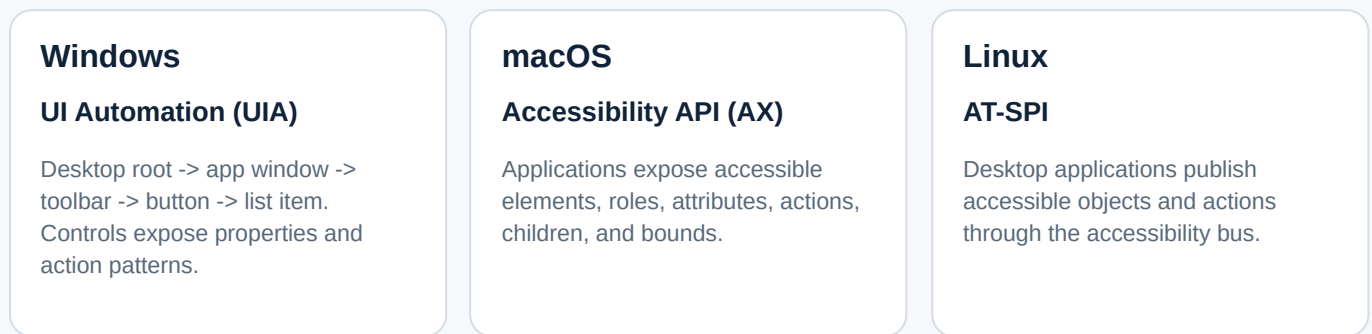
Fixed coordinates, template matching, color probes, blind Tab loops, timed sleeps. Fragile under layout, scale, theme, or latency changes.

Use: narrow stable targets only

Important: the same tool can move tiers. A keyboard action with no state check is D-tier. The same key sequence wrapped with preconditions, focus checks, output capture, and verification is B-tier and can approach A-tier reliability.

The accessibility tree is the desktop DOM

Desktop applications expose a semantic hierarchy to assistive technology and automation clients. Names differ by platform, but the useful fields are similar.



Representation	What it tells you	What it does not tell you
DOM	Web structure, attributes, text, events	Non-browser desktop UI
A11y tree	Role, label, state, actions, bounds	Unexposed canvas pixels
Screenshot	What the user can visually see	Meaning, stable identity, action contract
Coordinates	Where to send pointer input	What is there or whether it moved

Memory rule: the tree says *what*; the screenshot shows *appearance*; grounding links meaning to location; coordinates say *where to act*.



Turn common computer work into typed commands

Opening and closing apps is valid CLI work, but it is only the lifecycle layer. Real coverage comes from combining lifecycle control with app capabilities and verification.

1. Lifecycle

Open, focus, list, status, quit, restart. Implement once behind a cross-platform `appctl` adapter.

2. Capability

Use native CLIs, APIs, document formats, scripting bridges, plugins, or generated CLI-Anything harnesses.

3. Evidence

Read process state, exit code, JSON output, changed files, app state, or an accessibility-tree event.

Cross-platform lifecycle examples

OS	Open	Quit	Observe
macOS	<code>open -a "App"</code>	<code>osascript ... quit</code>	<code>pgrep -fl App</code>
Linux	<code>gtk-launch APP_ID</code>	<code>pkill -TERM name</code>	<code>pgrep -af name</code>
Windows	<code>Start-Process app</code>	<code>Stop-Process -Name app</code>	<code>Get-Process app</code>

Wrap operating-system commands behind one schema. Do not let the model invent raw shell text.

CLI-Anything's role

Use it to create or adopt a stateful command-line harness for software whose useful functions are otherwise trapped behind a graphical interface.

A harness should offer discoverable commands, structured JSON output, undo where possible, and tests against the real application.

What it does not solve

Launching a process is not the same as controlling its document, scene, timeline, or settings. Some apps expose no clean backend, and generated wrappers still require maintenance and verification.

```
appctl open obs appctl status obs --json appctl focus obs cli-anything-obs --json scene switch --name "Camera" appctl verify obs --expect-window "OBS Studio"
```

Target: take the workflows that are eligible for structured control and drive that class toward 100% coverage. The "40%" is a hypothesis until your own benchmark proves it.

Vim is not the problem. Blindness is.

A keyboard sequence becomes agent-grade when it has a contract. Treat it like an n8n node: typed inputs, declared preconditions, bounded execution, observable outputs, and a recovery path.

Keyboard node = macro + focus + state + timeout + proof

Strong keyboard pipeline

- **Named intent:** `editor.save_all`
- **Precondition:** editor is foreground; no modal dialog
- **Input:** file path or buffer selector
- **Sequence:** validated keys, not free-form text
- **Postcondition:** dirty flag cleared or file timestamp changed
- **Recovery:** Escape, refocus, retry once, then escalate

Weak keyboard pipeline

- Assumes the correct window has focus
- Uses a fixed number of Tab presses
- Sleeps for arbitrary durations
- Cannot detect a popup or changed layout
- Repeats destructive keys after uncertainty
- Reports success because keys were sent

Node contract

```
{
  "id": "vim.apply_refactor",
  "inputs": {"file": "string", "macro": "approved_id"},
  "requires": ["vim_mode=normal", "buffer_loaded"],
  "action": {"keys": ["@r"]},
  "expect": ["buffer_changed", "diagnostics_not_increased"],
  "timeout_ms": 3000,
  "on_fail": ["escape", "capture_state", "escalate"]
}
```

Where keyboard nodes shine

Editors

Vim motions, command palettes, save/build/test loops, repeatable refactors.

Terminal multiplexers

Create panes, route commands, capture pane output, attach to named sessions.

Stable applications

Known shortcuts for new/open/export, provided state can be checked before and after.

Design rule: use keys as an actuator, not as perception. Read focus and state from the accessibility tree, process table, editor API, filesystem, or captured terminal output.

Make the small model choose, not hallucinate

A tiny local language model can route intent, fill arguments, rank elements, and select recovery steps. It should not generate unconstrained shell commands or invent coordinates.



Good tiny-model job	Output shape	Why it fits
Choose a lane	{lane, confidence}	Small label set
Select a tool	{tool_id}	Catalog retrieval
Fill arguments	{app, action, target}	Constrained extraction
Rank UI elements	{element_id}	Short candidate list
Choose recovery	{retry fallback ask}	Finite policy

Model profile to benchmark

- 0.5B to 0.8B parameters
- 4-bit quantization where quality holds
- Grammar-constrained JSON output
- Short context built from retrieved tool specs
- Fast enough to run on every action

Under 1 GB should be treated as a disk/RAM budget to verify on the actual runtime, not a quality guarantee.

Keep outside the model

- Permissions and destructive-action gates
- Shell quoting and process management
- Path normalization and file checks
- Retries, timeouts, and idempotency
- Evidence collection and audit logs

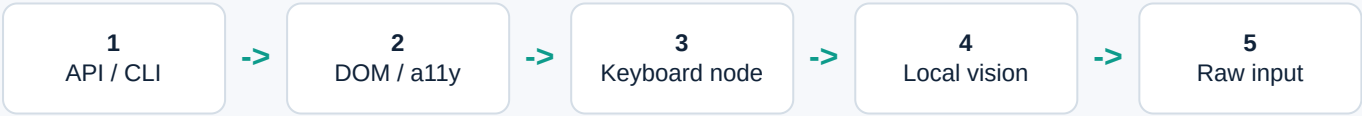
Browser savings: route browser tasks through API or DOM extraction first. Let the local model map intent to a known action. Use local OCR or a local vision model only when the page hides semantics, and reserve paid remote vision for unsupported edge cases.

One loop, five lanes, explicit escalation

The planner emits one typed action at a time. Every action must name an expected state change. Failure moves down one lane; it never triggers an unbounded retry loop.

Stage	Responsibility	Output
1. Observe	Collect process list, active window, DOM/a11y tree, screenshot, terminal output	Raw state bundle
2. Normalize	Reduce to app, role, name, value, state, bounds, confidence	Compact state
3. Route	Choose API/CLI, semantic tree, keyboard node, local vision, or raw input	Lane + tool candidates
4. Plan	Select one action and define its expected effect	Typed action
5. Validate	Check schema, allowlist, arguments, permissions, and risk	Approved or blocked
6. Act	Run exactly one adapter, node, or actuator	Execution result
7. Verify	Re-observe and compare actual state with expected state	Evidence verdict
8. Recover	Retry safely, switch lane, restore state, or ask a human	Next action
9. Learn	Record trace, failure cause, and reusable workflow candidate	Benchmark data

Lane ladder



```
{
  "goal": "save the current project",
  "lane": "keyboard_node",
  "tool": "editor.save_all",
  "args": {},
  "expect": {"dirty_buffers": 0},
  "risk": "low",
  "evidence": ["editor_state", "file_mtime"]
}
```

Local vision package

Screenshot crop -> OCR -> icon/text candidates -> local grounding model -> element ID or coordinates -> click -> screenshot/a11y diff.

Hard stop conditions

Unknown destination, destructive action without approval, repeated mismatch, low-confidence target, stale screenshot, unexpected dialog, or state that cannot be verified.

Perfect a bounded workflow set before chasing universality

Phase 1: inventory

- Log 50 real computer tasks
- Split each into atomic actions
- Mark current best lane
- Count frequency and failure cost

Phase 2: adapters

- Build cross-platform `appctl`
- Register existing CLIs and APIs
- Add CLI-Anything harnesses selectively
- Require JSON and evidence hooks

Phase 3: keyboard nodes

- Package Vim/tmux/editor macros
- Declare focus preconditions
- Add postcondition checks
- Test interruption and recovery

Phase 4: semantic + vision

- Add OS accessibility adapters
- Use local OCR before vision
- Benchmark a sub-1GB router
- Add local visual grounding fallback

Coverage scorecard

Metric	Definition	Desired direction
Semantic coverage	% actions completed without pixels	Up
Eligible-lane coverage	% CLI-suitable workflows fully supported	Toward 100%
Verified success	% actions with confirmed postcondition	Up
Fallback rate	% actions escalating to vision	Down
Recovery rate	% failures recovered without unsafe retry	Up
Local ratio	% decisions completed without paid inference	Up

Benchmark method: run the same fixed task set after every change. Record lane, latency, model memory, success, proof, retries, and human intervention. This turns "40%" from a guess into a baseline and shows whether your command lane is actually approaching complete coverage.

The five-second routing test

1. **Can a command or API do it?** Use that. Wrap it behind a typed tool.
2. **Can the DOM or accessibility tree name it?** Select the semantic element and invoke its action.
3. **Is there a stable keyboard workflow?** Call a verified macro node with focus and postconditions.
4. **Are semantics missing?** Use local OCR and local visual grounding.
5. **Only coordinates remain?** Click once, re-observe immediately, and stop on mismatch.

Core vocabulary

Observer: collects machine and UI state.

Normalizer: turns platform-specific state into one schema.

Router: chooses the cheapest reliable lane.

Grounder: maps an intended target to an element or location.

Actuator: sends a command, action, key, or pointer event.

Verifier: proves that the intended state changed.

Architecture in one line

```
goal -> state -> route -> typed action  
-> guard -> execute -> evidence ->  
recover
```

What to build first

1. `appctl` for open/focus/status/quit
2. Tool registry with JSON schemas
3. Vim/tmux macro-node runner
4. Windows/macOS/Linux a11y adapters
5. Small local router benchmark
6. Local OCR and grounding fallback

Sources and further reading

Microsoft UI Automation Tree Overview. Describes a desktop-rooted tree whose descendants represent application windows and controls.

<https://learn.microsoft.com/en-us/windows/win32/winauto/uiauto-treeoverview>

W3C Core Accessibility API Mappings 1.2. Maps web semantics across UIA, AT-SPI, and macOS AX APIs.

<https://www.w3.org/TR/2023/CRD-core-aam-1.2-20230626/>

freedesktop.org AT-SPI2. Technical background for Linux accessibility infrastructure.

<https://www.freedesktop.org/wiki/Accessibility/AT-SPI2/>

HKUDS CLI-Anything. Framework and catalog for stateful CLI harnesses around software applications.

<https://github.com/hkuds/cli-anything>

Final principle: build an owner-controlled computer-use system as a federation of small reliable tools. The model chooses among them; the operating system and application adapters do the work; verification decides whether the work actually happened.